```
            raise ValidationError(
                "Band field required when seeking a musician")
        if self.musician is not None:
            raise ValidationError(
                ("Musician field should be empty for a band "
                "seeking a musician"))
    else:
        # Validate seeking band case
        if self.musician is None:
            raise ValidationError(
                "Musician field required when seeking a band")
        if self.band is not None:
            raise ValidationError(
                ("Band field should be empty for a musician "
                "seeking a band"))

    super().clean()
```

When seeking a band, ensure the musician field gets populated and the band field is empty.

WARNING `.clean()` is not called automatically!

Early versions of Django did not include object validation methods, like `.clean()`, and to maintain backward compatibility, it was decided not call it automatically. It is possible to create an object programmatically that violates your validation code. The `.clean()` method gets called when you use the Django Admin, but if you open up the REPL and create an object by hand, `.clean()` is not called. One work around for this is to override the model's `.save()` method and force a call to `.clean()` inside it. The downside of this approach is `.save()` doesn't know what to do with a `ValidationError`, and if your code doesn't catch it, the error will show as a `500` error in your logs. If you do decide to override `.save()`, make sure you use a `try` block in your code to properly handle any `ValidationError` that gets raised.

7.2.2 A ModelAdmin for SeekingAd using Truncate

To test your new `SeekingAd` model out, you need an Admin class to go with it. Inside RiffMates/content/admin.py, add a `SeekingAdAdmin` class similar to those you wrote for the `bands` app. The Django Admin automatically uses the labels from the `seeking` field, so when you include it in `list_display`, it will show `"Musician"` rather than `"M"`.

The ad text from the object might be quite long; displaying it on the Django Admin summary page could be unwieldy. There are several template filters that perform *truncation* on strings. The `{{ |truncatewords }}` filter truncates a string on word boundaries. It would be nice to use this functionality outside of templates. If you go digging through the Django source code, you will find the `Truncator` class inside `django.utils.text`. This undocumented feature is what the truncation filters got built on top of. It is unclear to me why this isn't documented; it is a handy class to take advantage of outside a template. The `SeekingAdAdmin` class uses `Truncate` to show a short version of the ad description in the Django Admin listing page. The complete code for RiffMates/content/admin.py is shown in the following listing.